

Predictive Analytics Using Historical Data

Regression-based forecasting pipeline · Linear Regression · scikit-learn · pandas · matplotlib

● R² Score: 85.39%

● 120 Records Processed

● 10-Step Forecast

MODEL EVALUATION METRICS

R² SCORE

85.39%

Model performance: Good
✓

MAE

27.11

Mean Absolute Error

RMSE

33.57

Root Mean Squared Error

MSE

1127.0

Mean Squared Error

FUTURE FORECAST — NEXT 10 PERIODS

STEP	PERIOD	PREDICTED SALES	TREND ALIGNMENT
1	2034-Jan	491.2671	+25.4% vs last
2	2034-Feb	493.5810	+26.0% vs last
3	2034-Mar	495.8950	+26.6% vs last
4	2034-Apr	498.2089	+27.2% vs last
5	2034-May	500.5228	+27.8% vs last
6	2034-Jun	502.8367	+28.4% vs last
7	2034-Jul	505.1506	+29.0% vs last
8	2034-Aug	507.4645	+29.5% vs last
9	2034-Sep	509.7784	+30.1% vs last
10	2034-Oct	512.0923	+30.7% vs last

CONSOLE RUNTIME LOG

PREDICTIVE ANALYTICS USING HISTORICAL DATA

Regression-Based Forecasting System

Enter choice: 8

Loading dataset...
[WARNING] 'historical_data.csv' not found. Generating sample data...
[INFO] Sample dataset created → 'historical_data.csv' (122 rows)
[OK] Dataset loaded: 122 rows × 5 columns

Date	Sales	Marketing_Spend	Customer_Count	Region_Code
2014-01-01	207.45	67.91	224	1
2014-02-01	220.45	51.66	241	3
2014-03-01	249.40	75.54	256	1
2014-04-01	270.41	48.25	266	4
2014-05-01	241.21	68.89	228	2

Preprocessing data...

— Before Cleaning —

Rows : 122
Missing values: 6
Duplicates : 2

— After Cleaning —

Rows : 120
Missing values: 0
Duplicates : 0

DATASET INFORMATION

Date	datetime64[us]
Sales	float64
Marketing_Spend	float64
Customer_Count	int64
Region_Code	int64
Time_Index	int64

SUMMARY STATISTICS

	Sales	Marketing_Spend	Customer_Count	Region_Code	Time_Index
count	120.00	120.00	120.00	120.00	120.00
mean	350.16	105.74	420.18	2.41	59.50
std	89.05	28.57	106.73	1.21	34.79
min	178.48	48.25	224.00	1.00	0.00
25%	280.80	85.12	328.25	1.00	29.75
50%	353.59	104.74	429.00	2.00	59.50
75%	425.53	130.16	519.00	4.00	89.25
max	541.82	167.66	605.00	4.00	119.00

Target : Sales
Features : ['Marketing_Spend', 'Customer_Count', 'Region_Code', 'Time_Index']

Training Linear Regression model...
[OK] Model trained successfully.

MODEL EVALUATION METRICS

Mean Absolute Error (MAE) : 27.1050
Mean Squared Error (MSE) : 1126.9775
Root MSE (RMSE) : 33.5705
R² Score : 0.8539 (85.39%)

Model Performance : Good ✓

FUTURE FORECAST – Next 10 Periods

Step	Predicted Sales
------	-----------------

1	491.2671
2	493.5810
3	495.8950
4	498.2089
5	500.5228
6	502.8367
7	505.1506
8	507.4645
9	509.7784
10	512.0923

Generating visualizations...
[OK] Charts saved → 'analytics_dashboard.png'

✓ Full pipeline complete!

Enter choice: 0

Goodbye! ✓

PYTHON IMPLEMENTATION PIPELINE

predictive_analytics.py

```
"""
=====
Predictive Analytics Using Historical Data
Author: Predictive Analytics System
Description: A complete predictive analytics pipeline
            using Linear Regression on historical data.
=====
"""

import os
import sys
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
import warnings
warnings.filterwarnings("ignore")

DATASET_FILE = "historical_data.csv"

def generate_sample_dataset():
    np.random.seed(42)
    n = 120
    dates = pd.date_range(start="2014-01-01", periods=n, freq="MS")
    trend = np.linspace(200, 500, n)
    seasonal = 40 * np.sin(2 * np.pi * np.arange(n) / 12)
    noise = np.random.normal(0, 15, n)
    sales = trend + seasonal + noise

    df = pd.DataFrame({
        "Date": dates,
        "Sales": np.round(sales, 2),
        "Marketing_Spend": np.round(trend * 0.3 + np.random.normal(0, 10, n), 2),
        "Customer_Count": np.round(trend * 1.2 + np.random.normal(0, 20, n)).astype(int),
        "Region_Code": np.random.choice([1, 2, 3, 4], n),
    })
    df.to_csv(DATASET_FILE, index=False)

def preprocess(df):
    df = df.drop_duplicates()
    num_cols = df.select_dtypes(include=[np.number]).columns
    for col in num_cols:
        df[col] = df[col].fillna(df[col].median())
    df["Time_Index"] = np.arange(len(df))
    return df

def build_model(df, target, features):
    X = df[features].values
    y = df[target].values
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
    X_train, X_test, y_train, y_test = train_test_split(
        X_scaled, y, test_size=0.2, random_state=42)
    model = LinearRegression()
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
```

